“A COMPARATIVE AND EXPLAINABLE MACHINE LEARNING FRAMEWORK FOR DERMATOLOGY MULTICLASS CLASSIFICATION”

A PROJECT REPORT SUBMITTED TO BHARATHIAR UNIVERSITY IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF DATA SCIENCE

Submitted By

SWATHI S

(Reg. No. 2328M0119)

Under the Guidance of

Dr. R. KARTHIK MCA., PhD.,

ASSISTANT PROFESSOR

DEPARTMENT OF DATA SCIENCE

SRI KRISHNA ADITHYA COLLEGE OF ARTS & SCIENCE

Recognized by UGC with 2(f) & 12(B) status | Affiliated to Bharathiar University

Kovaipudur, Coimbatore - 641042

SRI KRISHNA INSTITUTIONS

MARCH 2026

CERTIFICATE

This is to certify that the project work entitled **"A COMPARATIVE AND EXPLAINABLE MACHINE LEARNING FRAMEWORK FOR DERMATOLOGY MUTICLASS CLASSIFICATION",** is a bona-fide work done by **SWATHI S (2328M0119)** during the period of **2025-2026** of her study in the Department of Data Science at Sri Krishna Adithya College of Arts and Science, Coimbatore, Affiliated to Bharathiar University, in partial fulfillment of requirements for the award of the degree of Bachelor of Science in Data Science under my supervision and guidance.The project work is an original work of the candidate and to the best of my knowledge has not been submitted, in part or in full, for any Diploma / Degree / Associateship / Fellowship or other similar titles in this or any other University. No portion of the dissertation is a reproduction from any other source, published or unpublished, without acknowledgement.

SIGNATURE OF THE GUIDE

SIGNATURE OF THE HOD **SIGNATURE OF THE PRINCIPAL**

Viva Voce Conducted on:_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I, **S. SWATHI** hereby declare that the project work entitled **"A COMPARATIVE AND EXPLAINABLE MACHINE LEARNING FRAMEWORK FOR DERMATOLOGY MULTICLASS CLASSIFICATION"** submitted to Sri Krishna Adithya College of Arts and Science, Coimbatore Affiliated to Bharathiar University in partial fulfillment of the requirements for the award of the Degree of **BACHELOR OF SCIENCE IN DATA SCIENCE** is a record of original work done by me during 2025 – 2026 under the supervision and guidance of **Dr. R. KARTHIK, M.C.A., Ph.D.,** The report has not been submitted for the award of any Degree / Diploma / Associate ship / Fellowship.

PLACE: COIMBATORE SIGNATURE OF THE CANDIDATE

DATE: **(SWATHI S)**

ACKNOWLEDGEMENT

First and foremost, I thank the almighty through whom all things are possible. My work would not be possible without the gifts god gave me.

The success of the project depends upon the efforts invested. It is my duty to acknowledge and thank the individuals who has contributed towards the successful completion of my dissertation.

I would like to express my whole-hearted thanks to **Smt. S. MALARVIZHI**, **Chairperson & Managing Trustee**, Sri Krishna Institutions, Coimbatore.

I take this opportunity to express my profound and whole hearted thanks to our Principal **Dr. S. PALANIAMMAL, M.Sc., M.Phil., Ph.D.,** Sri Krishna Adithya College of Arts and Science for reinforcing my effort with support and guidance.

I sincerely thank my esteemed Dean **Dr. K. GEETHA, M.Sc., M.Phil., UGC – SET, Ph. D.,** Associate Professor, for their valuable support and guidance throughout this project.

I sincerely thank my esteemed Head of the Department **Dr. S. THILAGAVATHI, MCA, M.Phil., Ph.D., MBA,** Department of Data Science, Sri Krishna Adithya College of Arts and Science for their valuable support and guidance throughout this project.

I owe my heart and profound gratitude to my internal guide **Dr. R. KARTHIK, M.C.A., Ph.D.,** Assistant Professor, Department of Data Science, Sri Krishna Adithya College of Arts and Science, Coimbatore for giving continuous encouragement to complete this work. This project would not be a success for me without their valuable comments and suggestions.

The successful completion of my project would not have been possible without my parent's support. I take this opportunity to thank them for their continuous encouragement. I convey my thankfulness to all my friends who were with me to share my happiness and agony.

SWATHI S

CONTENTS

CHAPTER NO		TITLE	PAGE NO
	CERTIFICATE		**I**
	DECLARATION		**II**
	ACKNOWLEDGEMENT		**III**
	ABSTRACT		**1**
1	**INTRODUCTION**		**2**
	1.1	Introduction	2
	1.2	Project Workflow	3
	1.3	Data Overview	4
2	**SYSTEM STUDY**		**7**
	2.1	Existing System	7
	2.2	Proposed System	8
	2.3	System Specification	9
3	**SYSTEM DEVELOPMENT**		**11**
	3.1	Importing Libraries	11
	3.2	Data Loading	12
	3.3	Data Preprocessing	13
	3.4	EDA	17
	3.5	Model Development	28
	3.6	Model Evaluation	32
	3.7	Model Optimization	36
	3.8	Model Testing	37
	3.9	Model Comparison	44
	3,10	Results	46

4	**STREAMLIT WEB INTERFACE**		**48**
	4.1	Dashboard module	48
	4.2	Prediction module	49
	4.3	Explainability AI Module	49
5	**CONCLUSION**		
	5.1	Conclusion	**51**
	5.2	Future Enhancement	52
	5.3	Reference	53
	5.4	Bibliography	54

ABSTRACT

The rapid advancement of Artificial Intelligence and Machine Learning has significantly transformed various domains, particularly the healthcare sector, where early diagnosis and accurate disease classification play a critical role in improving patient outcomes. Dermatological diseases are among the most common medical conditions worldwide, and accurate diagnosis often requires careful analysis of multiple clinical attributes. However, manual diagnosis can be time-consuming and subject to human variability. In this context, the present study focuses on developing a machine learning–based dermatological disease classification system using structured clinical data to assist in diagnostic decision-making. The project involves the implementation and comparative analysis of multiple supervised machine learning algorithms, including Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, and Naïve Bayes. The dataset used in this study contains various clinical features relevant to dermatological conditions. A systematic data preprocessing pipeline was implemented, which included handling missing values, encoding categorical attributes, and applying feature scaling techniques to ensure optimal model performance. The dataset was divided into training and testing subsets to evaluate the generalization capability of the models.

Each algorithm was trained and evaluated using standard performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis. Comparative results demonstrated that ensemble-based methods, particularly Random Forest, achieved superior predictive performance due to their ability to capture complex non-linear relationships among clinical features while reducing overfitting. The findings confirm that machine learning techniques can effectively identify patterns within dermatological data and provide reliable classification results under controlled experimental conditions. Although the proposed system is not intended to replace dermatologists, it serves as a supportive decision-making tool that can enhance diagnostic efficiency and assist healthcare professionals in early-stage disease identification. The study also discusses limitations related to dataset diversity, absence of image-based analysis, and lack of real-world clinical validation, while suggesting potential enhancements such as multimodal integration and advanced hyperparameter optimization.

1.1.INTRODUCTION

The healthcare industry has witnessed significant technological advancements in recent years, particularly with the integration of Artificial Intelligence (AI) and Machine Learning (ML) into diagnostic and decision-support systems. Among various medical specialties, dermatology presents a unique opportunity for AI-driven solutions due to the structured nature of clinical attributes and the pattern-based characteristics of many skin diseases. Dermatological conditions are among the most prevalent health issues worldwide, affecting individuals across all age groups and demographics. Early and accurate diagnosis is essential to prevent disease progression, reduce complications, and improve patient quality of life. However, traditional diagnostic procedures often rely heavily on clinical expertise, experience, and visual examination, which may lead to variability in diagnosis and increased workload for specialists. Machine learning techniques offer a promising approach to addressing these challenges by enabling computers to learn patterns from historical medical data and make predictive classifications. Supervised learning algorithms, in particular, have demonstrated strong performance in medical data analysis by identifying relationships between input clinical features and corresponding disease categories. By training models on structured datasets containing dermatological attributes, it becomes possible to develop predictive systems that assist healthcare professionals in making faster and more consistent diagnostic decisions.

The primary objective of this project is to design and implement a machine learning–based dermatological disease classification system using structured clinical data. The study involves the application and comparative evaluation of multiple supervised learning algorithms.A systematic methodology is followed, beginning with data preprocessing steps such as handling missing values, encoding categorical variables, and performing feature scaling to enhance model performance. The dataset is then divided into training and testing sets to evaluate the generalization ability of each model. Through comparative analysis using evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix, the project aims to identify the most effective algorithm for dermatological disease classification. By exploring different machine learning techniques and analyzing their strengths and limitations, this study contributes to understanding how AI-driven systems can support clinical decision-making in dermatology.

1.2 PROJECT WORKFLOW

This project aims to develop an intelligent dermatological disease classification system using machine learning techniques applied to structured clinical data. Accurate and early identification of skin diseases is essential for effective treatment and prevention of complications. Traditional diagnosis relies heavily on clinical expertise and manual evaluation of symptoms, which may sometimes lead to variability in interpretation. By integrating Exploratory Data Analysis (EDA) and predictive modelling techniques, this project seeks to build a reliable decision-support system that can assist healthcare professionals in diagnosing dermatological conditions more efficiently and consistently. The combination of systematic data preprocessing, multiple machine learning algorithms, and comparative performance analysis ensures a comprehensive and scientifically structured approach toward disease classification.

Fig 1.1.Project Workflow

1.3 DATA OVERVIEW

The dataset used in this project is the **Dermatology dataset**, which contains structured clinical attributes related to various dermatological diseases. The dataset consists of multiple patient records, where each record represents a set of clinical features observed during medical examination, along with a corresponding disease class label. This dataset is widely used for academic research in machine learning–based medical diagnosis due to its well-defined features and multi-class classification nature.

The dataset contains **34 input attributes** and **1 target variable (class label)**. The input features represent clinical and histopathological characteristics associated with skin diseases. These attributes include erythema, scaling, definite borders, itching, koebner phenomenon, polygonal papules, follicular papules, oral mucosal involvement, knee and elbow involvement, scalp involvement, family history, and other medical indicators. Most of these features are measured on a numerical scale indicating severity levels, while some are binary in nature. One attribute, age, is continuous and represents the patient's age in years.

The target variable (class attribute) represents six different dermatological disease categories. These include various erythemato-squamous diseases that often share similar clinical features, making classification a challenging multi-class problem. The presence of overlapping symptoms among disease classes increases the complexity of prediction and highlights the need for effective machine learning algorithms.

CLINICAL FEATURES (1-12):

1. Age – Age of the patient in years (continuous numerical value).
2. Erythema – Degree of redness of the skin caused by increased blood flow in superficial capillaries (graded scale).
3. Scaling – Presence and severity of flaky skin or shedding of the outer skin layer.
4. Definite borders – Whether the skin lesion has clearly defined boundaries separating it from surrounding skin.
5. Itching – Intensity of pruritus (itching sensation) experienced by the patient.

6. Koebner phenomenon – Appearance of new skin lesions on areas of trauma or injury.
7. Polygonal papules – Presence of flat-topped, polygon-shaped raised lesions.
8. Follicular papules – Presence of small raised bumps associated with hair follicles.
9. Oral mucosal involvement – Involvement of lesions inside the oral cavity.
10. Knee and elbow involvement – Presence of lesions specifically on knees and elbows.
11. Scalp involvement – Presence of dermatological symptoms affecting the scalp region.
12. Family history – Indicates whether there is a family history of similar dermatological disease (binary feature).

HISTOPATHOLOGICAL FEATURES:

13. Melanin incontinence – Leakage of melanin pigment into the dermis due to damage in basal cells.
14. Eosinophils in the infiltrate – Presence of eosinophil white blood cells in the inflammatory infiltrate.
15. PNL infiltrate – Presence of polymorphonuclear leukocytes in the inflammatory region.
16. Fibrosis of the papillary dermis – Thickening or scarring in the upper dermal layer.
17. Exocytosis – Migration of inflammatory cells into the epidermis.
18. Acanthosis – Thickening of the epidermal layer of the skin.
19. Hyperkeratosis – Thickening of the outermost layer of the skin (stratum corneum).
20. Parakeratosis – Retention of nuclei in the stratum corneum indicating abnormal keratinization.
21. Clubbing of the rete ridges – Bulbous enlargement of epidermal projections into the dermis.
22. Elongation of the rete ridges – Increased length of epidermal projections extending into the dermis.
23. Thinning of the suprapapillary epidermis – Reduced thickness of the epidermal layer above dermal papillae.
24. Spongiform pustule – Formation of pus-filled spaces within the epidermis.
25. Munro microabscess – Small collections of neutrophils in the stratum corneum.
26. Focal hypergranulosis – Localized thickening of the granular layer of the epidermis.
27. Disappearance of the granular layer – Loss of the granular cell layer in certain skin regions.

28. Vacuolisation and damage of basal layer – Structural damage and vacuole formation in the basal cell layer.
29. Spongiosis – Intercellular edema within the epidermis.
30. Saw-tooth appearance of rete ridges – Irregular zig-zag pattern of epidermal ridges.
31. Follicular horn plug – Accumulation of keratin blocking hair follicles.
32. Perifollicular parakeratosis – Abnormal keratinization around hair follicles.
33. Inflammatory mononuclear infiltrate – Presence of mononuclear inflammatory cells in tissue.
34. Band-like infiltrate – Dense, band-shaped inflammatory cell infiltration in the dermis.

TARGET ATTRIBUTE

35. Class (Disease Type)

The class attribute represents six dermatological diseases:

- Psoriasis
- Seborrheic dermatitis
- Lichen planus
- Pityriasis rosea
- Chronic dermatitis
- Pityriasis rubra pilaris

2. SYSTEM STUDY

The system study is an important phase in the development of the project as it provides a clear understanding of the current methods used for dermatological disease diagnosis and highlights the need for an improved automated solution. It involves the analysis of the existing system, identification of its limitations, and the design of a proposed system that overcomes these drawbacks using machine learning techniques. This study helps in defining the functional and technical requirements of the new system and ensures that the developed model is efficient, accurate, and reliable.

2.1 EXISTING SYSTEM

Dermatological diseases are traditionally diagnosed through manual clinical examination and histopathological analysis by experienced dermatologists. This process relies on visual inspection of symptoms and microscopic tissue evaluation. Although effective, it is highly dependent on practitioner expertise and can be difficult when diseases have overlapping symptoms.

With advancements in Artificial Intelligence, machine learning models such as Decision Trees, Support Vector Machines, and Random Forest have been applied for disease classification. However, many existing systems focus mainly on achieving high accuracy using limited algorithms. They often rely on simple train–test splits, use default hyperparameters, and evaluate performance primarily through accuracy, with limited attention to validation, optimization, or feature interpretability. As a result, these systems lack comprehensive analytical depth and structured comparison.

The manual diagnostic approach is time-consuming, costly, and heavily dependent on dermatologist expertise, which may lead to inconsistent results. Limited specialist availability can also delay diagnosis and treatment.

Existing machine learning systems also have limitations. Many rely on a single model without proper comparative analysis or robust validation techniques such as cross-validation, increasing the risk of overfitting. Hyperparameters are often not optimized, leading to suboptimal performance. Additionally, evaluation is usually restricted to accuracy alone, which is insufficient for medical diagnosis problems.

Feature analysis and interpretability are frequently neglected, reducing transparency and clinical trust. Moreover, many systems are not designed as complete end-to-end frameworks with structured preprocessing, visualization, and detailed evaluation. These limitations highlight the need for a more reliable, optimized, and analytically validated dermatological disease classification system.

2.2 PROPOSED SYSTEM

The proposed system is a comprehensive machine learning–based framework for dermatological disease classification designed to overcome the limitations of existing approaches. It performs structured analysis on a dermatology dataset containing clinical and histopathological features across six disease classes. The system emphasizes not only prediction accuracy but also reliability, comparison, and interpretability.

The process begins with data preprocessing, including handling missing values, correcting data types, and preparing the dataset for analysis. Exploratory Data Analysis (EDA) is conducted to study feature distributions, class balance, and correlations among variables. Multiple supervised algorithms—such as Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, Gradient Boosting, and Support Vector Machine—are trained and compared. Stratified K-Fold Cross-Validation ensures reliable evaluation, and Grid Search is used to optimize the best-performing model.

Performance is assessed using multiple metrics, including Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. Feature importance analysis is also performed to identify the most influential attributes. By combining validation, optimization, and interpretability, the proposed system offers a structured, reliable, and data-driven approach to dermatological disease classification.

The proposed system includes a structured preprocessing pipeline to ensure clean and well-formatted data for model training. It incorporates detailed Exploratory Data Analysis to understand feature relationships and disease patterns using statistical and visualization techniques.

A key feature is the comparative analysis of multiple machine learning algorithms rather than relying on a single model. Stratified K-Fold Cross-Validation is used to ensure consistent and unbiased performance evaluation, while Grid Search optimizes hyperparameters for improved predictive accuracy.

The system also evaluates models using multiple performance metrics beyond accuracy, providing a more complete assessment. Additionally, feature importance analysis enhances interpretability by identifying the most influential clinical and histopathological attributes. Overall, the framework is reliable, scalable, and analytically strong, supporting accurate and transparent dermatological disease classification.

2.3 SYSTEM SPECIFICATION

The proposed system is developed as a machine learning–based dermatological disease classification platform that analyzes structured clinical data and predicts the type of skin disease. The system is designed to assist in medical decision support by applying data preprocessing, exploratory data analysis, and multiple classification algorithms to achieve accurate and reliable predictions. The architecture of the system is simple, scalable, and cost-effective, making it suitable for academic research as well as future enhancement into a real-time clinical support application.

2.3.1 HARDWARE SPECIFICATION

- Processor: Intel Core i3 / Intel Core i5 or higher
- RAM: Minimum 4 GB (Recommended: 8 GB for better performance)
- Storage: 256 GB HDD / SSD or higher
- System Type: Laptop / Desktop computer
- Input Source: Dermatology dataset (CSV file)
- Internet Connection: Required for using Google Colab

2.3.2 SOFTWARE SPECIFICATION

- Operating System: Windows 10 / 11, Linux, or macOS
- Programming Language: Python 3.x
- Development Environment: Jupyter Notebook / Google Colab / Visual Studio Code
- Data Processing Libraries: Pandas, NumPy
- Data Visualization Libraries: Matplotlib, Seaborn
- Machine Learning Library: Scikit-learn
- File Format Used: CSV (for dataset storage and processing)
- Web Browser: Google Chrome / Microsoft Edge

3.SYSTEM DEVELOPMENT

The system development phase focuses on the practical implementation of the dermatological disease classification framework using machine learning techniques. This phase involves transforming the designed workflow into a functional analytical model through a sequence of well-defined steps, including data handling, preprocessing, exploratory analysis, model building, validation, and performance comparison. The development is carried out in a Python-based environment using scientific computing and machine learning libraries, ensuring efficiency, scalability, and reproducibility of the results.

3.1 IMPORTING LIBRARIES

The project uses pandas for loading, cleaning, and preprocessing the PCOS dataset in a structured tabular format, while numpy supports numerical and mathematical computations. kagglehub is used to download the dataset directly from Kaggle, and **os** helps manage file paths and system-level operations. For data visualization, matplotlib creates basic plots such as histograms and performance graphs, whereas seaborn generates advanced statistical visualizations like heatmaps for correlation analysis. The sklearn.model_selection module is used for splitting the dataset, performing cross-validation, and tuning hyperparameters. sklearn.preprocessing handles feature scaling and categorical encoding. Model evaluation is carried out using sklearn.metrics, including accuracy, F1-score, classification report, and confusion matrix. Various machine learning algorithms are implemented using sklearn.ensemble (Random Forest and Gradient Boosting), sklearn.linear_model (Logistic Regression), sklearn.svm (SVC), sklearn.neighbors (KNN), and sklearn.tree (Decision Tree). Additionally, the warnings module suppresses unnecessary messages, and matplotlib style settings ensure clear and consistent visualizations.

CODE:

```
import pandas as pd import

numpy as np import kagglehub

import os import
```

```
matplotlib.pyplot as plt import

seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score,

GridSearchCV, StratifiedKFold from sklearn.preprocessing import

StandardScaler, LabelEncoder

from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, f1_score

from sklearn.ensemble import

RandomForestClassifier, GradientBoostingClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.svm import SVC from sklearn.neighbors

import KNeighborsClassifier from sklearn.tree

import DecisionTreeClassifier import warnings

warnings.filterwarnings('ignore')
```

3.2. DATA LOADING

Data loading is the process of importing and reading data from various sources into the working environment for analysis and model development. This step ensures that the dataset is successfully accessed, stored in an appropriate data structure, and made ready for preprocessing and further operations. The data is typically loaded from sources such as CSV files, Excel files, databases, or online repositories and is converted into a DataFrame format for easy manipulation.

In this project, the dermatology dataset is directly downloaded from Kaggle using the **kagglehub** library. This approach automates the dataset retrieval process and eliminates the need for manual downloading and file handling. The kagglehub.dataset_download() function is used to download the dataset from the Kaggle repository and store it in a local directory. The returned path represents the location where the dataset files are saved in the system.

The os.listdir(path) function is used to display the list of files available in the downloaded dataset directory. This helps in verifying that the dataset has been successfully downloaded and identifying the correct file to load.

The pd.read_csv() function from the pandas library is used to read the CSV file and convert it into a DataFrame.

- **df:**

  This variable contains the data from the file *dermatology_database_1.csv*. The dataset includes multiple clinical and histopathological features along with the class label representing different types of skin diseases. This DataFrame serves as the primary input for data preprocessing, exploratory data analysis, model training, and evaluation.

CODE:

```
path=kagglehub.dataset_download("olcaybolat1/dermatology-dataset-classifc
ation" print("Path to dataset files:", path) print(os.listdir(path)) df =
pd.read_csv(os.path.join(path,'dermatology_database_1.csv'))
```

3.3 DATA PREPROCESSING MODULE

3.3.1. FEATURE CATEGORIZATION

This code segment is used to organize the attributes of the dermatology dataset into meaningful groups based on their medical relevance. It first prints a heading to clearly indicate the start of the feature categorization section. Two separate lists are then created: one for **clinical features** and another for **histopathological features**.

The **clinical features** represent the visible symptoms observed on the patient's skin, such as erythema, scaling, itching, and scalp involvement. These features are typically identified through physical examination. The **histopathological features**, on the other hand, correspond to microscopic characteristics obtained through laboratory analysis of skin tissue, such as acanthosis, hyperkeratosis, spongiosis, and inflammatory infiltrates.

After defining these categories, the code prints the total number of clinical and histopathological features, along with the additional numerical feature **age** and the **target variable (class)**, which represents the six different types of skin diseases.

This categorization improves the interpretability of the dataset, helps in understanding the domain-specific structure of the features, and supports better exploratory data analysis and model development by distinguishing between symptom-based and laboratory-based attributes.

CODE :

```
print("\nFeature Categories:")
print("-" * 40)

clinical_features = ['erythema', 'scaling', 'definite_borders', 'itching', 'koebner_phenomenon',
                     'polygonal_papules', 'follicular_papules', 'oral_mucosal_involvement',
                     'knee_and_elbow_involvement', 'scalp_involvement', 'family_history']

histopathological_features = ['melanin_incontinence', 'eosinophils_infiltrate', 'PNL_infiltrate',
                              'fibrosis_papillary_dermis', 'exocytosis', 'acanthosis', 'hyperkeratosis',
                              'parakeratosis', 'clubbing_rete_ridges', 'elongation_rete_ridges',
                              'thinning_suprapapillary_epidermis', 'spongiform_pustule', 'munro_microabcess',
                              'focal_hypergranulosis', 'disappearance_granular_layer',
                              'vacuolisation_damage_basal_layer', 'spongiosis', 'saw_tooth_appearance_retes',
                              'follicular_horn_plug', 'perifollicular_parakeratosis',
                              'inflammatory_mononuclear_infiltrate', 'band_like_infiltrate']
```

OUTPUT :

```
...
    Feature Categories:
    ----------------------------------------
    Clinical Features (visible symptoms): 11
    Histopathological Features (microscopic): 22
    Other: age
    Target: class (1-6 representing 6 skin diseases)
```

3.3.2. MISSING VALUES ANALYSIS

This section focuses on identifying and analysing missing values in the dermatology dataset as part of the data preprocessing phase. The dataset represents missing values using the symbol '?' instead of standard null values, so the code first scans the entire DataFrame to count how many times this symbol appears in each column. Only columns containing missing values are displayed for clarity. In addition to checking for the custom symbol, the code also uses the isnull() function to detect standard null values (NaN) and prints their total count. This systematic

identification step is essential because machine learning models cannot handle missing data directly, and understanding the location and extent of missing values helps determine the appropriate cleaning strategy.

Special attention is given to the age attribute, as age is a significant feature for disease analysis and prediction. The code filters and displays only those records where the age value is '?' to clearly identify rows with missing age information, along with their corresponding class labels. By examining these records, it becomes easier to understand the potential impact of missing age data on classification outcomes. Identifying such records allows informed decisions about how to treat them, whether through imputation or other preprocessing techniques, ensuring that the dataset remains suitable for analysis.

To handle missing age values, a clean copy of the dataset is created to preserve the original data. The '?' symbols in the age column are replaced with NaN, and the column is converted from object (string) type to a numeric format to enable statistical operations. The median age is then calculated and used to fill the missing entries, as the median is robust to outliers and provides a reliable central value for skewed data. Finally, the code confirms the preprocessing by printing the median used, the number of missing values after imputation, and the updated data type, ensuring that the dataset is clean, consistent, and ready for further analysis and machine learning model training.

CODE:

```
print("=" * 60)
print("MISSING VALUES ANALYSIS")
print("=" * 60)

# Check for '?' which represents missing values in this dataset
missing_counts = (df == '?').sum()
missing_cols = missing_counts[missing_counts > 0]
print(f"\nColumns with '?' (missing values):")
print(missing_cols)

# Also check standard null values
null_counts = df.isnull().sum()
print(f"\nStandard null values: {null_counts.sum()}")
```

```
============================================================
MISSING VALUES ANALYSIS
============================================================

Columns with '?' (missing values):
age    8
dtype: int64

Standard null values: 0
```

```python
print("\nRows with missing age values:")
df[df['age'] == '?'][['age', 'class']]
```

```
Rows with missing age values:
```

	age	class
33	?	1
34	?	4
35	?	2
36	?	3
262	?	5
263	?	5
264	?	5
265	?	5

```python
df_clean = df.copy()

# Replace '?' with NaN, then convert age to numeric
df_clean['age'] = df_clean['age'].replace('?', np.nan)
df_clean['age'] = pd.to_numeric(df_clean['age'])

# Impute missing age with median (robust to outliers)
median_age = df_clean['age'].median()
df_clean['age'] = df_clean['age'].fillna(median_age)

print(f"Median age used for imputation: {median_age}")
print(f"Missing values after cleaning: {df_clean.isnull().sum().sum()}")
print(f"Age dtype after conversion: {df_clean['age'].dtype}")
```

```
Median age used for imputation: 35.0
Missing values after cleaning: 0
Age dtype after conversion: float64
```

3.4. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is a crucial step in the data analysis process that involves examining, summarizing, and visualizing the dataset to understand its main characteristics before applying machine learning algorithms. It helps in identifying patterns, detecting missing values, understanding class distribution, analyzing relationships between features, and spotting anomalies or outliers. Through statistical summaries and graphical representations such as bar charts, pie charts, histograms, box plots, and heatmaps, EDA provides meaningful insights into the structure of the data. This understanding enables better feature selection, appropriate preprocessing, and the selection of suitable machine learning models, ultimately improving the accuracy and reliability of the final system.

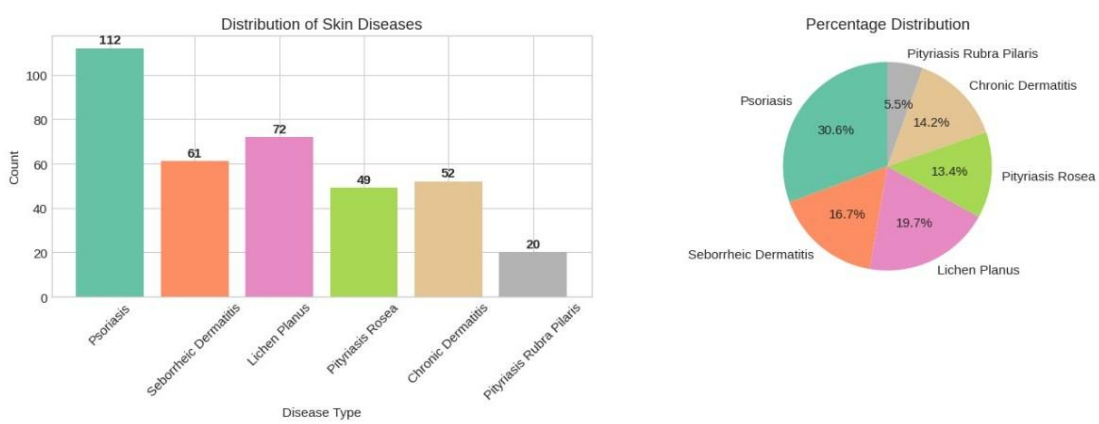
3.4.1. CLASS DISTRIBUTION ANALYSIS OF SKIN DISEASES

The code performs Exploratory Data Analysis (EDA) to examine the distribution of dermatological disease classes in the dataset. It begins by creating a dictionary called disease_names that maps numerical class labels (1–6) to their respective disease names, improving readability in outputs and visualizations. The value_counts() function is then applied to the target variable to compute the frequency of each class, and the results are sorted by class index to maintain logical order. To visually interpret the distribution, two plots are generated: a bar chart displaying the number of samples per disease category with count labels placed above each bar, and a pie chart illustrating the percentage contribution of each class to the overall dataset. Finally, a textual summary is printed showing both the total count and percentage for each class, providing a clear numerical confirmation of the class distribution. This process helps assess whether the dataset is balanced before proceeding to model training.

CODE:

```
disease_names = {
    1: 'Psoriasis',
    2: 'Seborrheic Dermatitis',
    3: 'Lichen Planus',
    4: 'Pityriasis Rosea',
    5: 'Chronic Dermatitis',
    6: 'Pityriasis Rubra Pilaris'
}

class_counts = df_clean['class'].value_counts().sort_index()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart
colors = plt.cm.Set2(np.linspace(0, 1, 6))
bars = axes[0].bar([disease_names[i] for i in class_counts.index], class_counts.values, color=colors)
axes[0].set_xlabel('Disease Type')
axes[0].set_ylabel('Count')
axes[0].set_title('Distribution of Skin Diseases')
axes[0].tick_params(axis='x', rotation=45)
for bar, count in zip(bars, class_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1, str(count),
                 ha='center', va='bottom', fontweight='bold')

# Pie chart
axes[1].pie(class_counts.values, labels=[disease_names[i] for i in class_counts.index],
            autopct='%1.1f%%', colors=colors, startangle=90)
axes[1].set_title('Percentage Distribution')

plt.tight_layout()
plt.show()

print("\nClass Distribution Summary:")
for idx, count in class_counts.items():
    print(f"  {disease_names[idx]}: {count} ({count/len(df_clean)*100:.1f}%)")
```

OUTPUT:



```
Class Distribution Summary:
  Psoriasis: 112 (30.6%)
  Seborrheic Dermatitis: 61 (16.7%)
  Lichen Planus: 72 (19.7%)
  Pityriasis Rosea: 49 (13.4%)
  Chronic Dermatitis: 52 (14.2%)
  Pityriasis Rubra Pilaris: 20 (5.5%)
```

Fig 3.4.1.Class Distribution

INSIGHTS:

- Psoriasis is the most common disease, accounting for the highest number of cases (around 30%).
- Lichen Planus and Seborrheic Dermatitis follow as the next most frequent conditions.
- Pityriasis Rosea and Chronic Dermatitis have moderate representation in the dataset.
- Pityriasis Rubra Pilaris is the least common, with a very small percentage of cases.
- The distribution is imbalanced, with a few diseases dominating the dataset while others are relatively rare.

3.4.2. AGE DISTRIBUTION ANALYSIS

The code performs Exploratory Data Analysis to understand the distribution of patient age and its relationship with different dermatological disease classes. First, a histogram is plotted to visualize the overall age distribution in the dataset, showing how patients are distributed across various age groups. Vertical dashed lines are added to represent the mean and median age, allowing comparison of central tendency and helping to determine whether the distribution is symmetric or skewed. Next, a box plot is generated to analyze age distribution separately for each disease class. This visualization displays the median, interquartile range, spread, and potential outliers for each category, enabling comparison of age variability across diseases. Together, these plots provide both overall and class-wise insights into how age influences dermatological disease patterns.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Overall age distribution
axes[0].hist(df_clean['age'], bins=20, edgecolor='black', alpha=0.7, color='steelblue')
axes[0].axvline(df_clean['age'].mean(), color='red', linestyle='--', label=f'Mean: {df_clean["age"].mean():.1f}')
axes[0].axvline(df_clean['age'].median(), color='green', linestyle='--', label=f'Median: {df_clean["age"].median():.1f}')
axes[0].set_xlabel('Age')
axes[0].set_ylabel('Frequency')
axes[0].set_title('Age Distribution of Patients')
axes[0].legend()

# Age by disease type
age_by_class = [df_clean[df_clean['class'] == i]['age'].values for i in range(1, 7)]
bp = axes[1].boxplot(age_by_class, labels=[disease_names[i][:10] for i in range(1, 7)], patch_artist=True)
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
axes[1].set_xlabel('Disease Type')
axes[1].set_ylabel('Age')
axes[1].set_title('Age Distribution by Disease Type')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

OUTPUT :

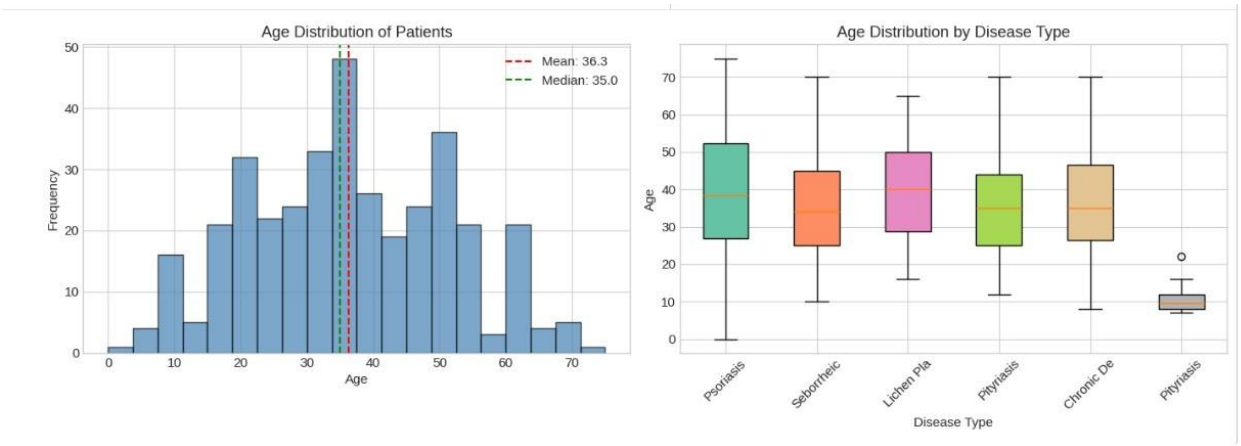


Fig 3.4.2.Age Distribution

INSIGHTS:

- The average age is around 36, with most patients falling in the 20–50 age range.
- The distribution is slightly right-skewed, indicating the presence of some older patients.
- Psoriasis and Chronic Dermatitis affect a wide range of age groups.
- Seborrheic Dermatitis is more common among younger adults, typically between 20 and 40 years.
- Lichen Planus tends to occur more in middle-aged individuals, around 40 years.
- Pityriasis shows a fairly even distribution across ages.
- Tinea is predominantly seen in children and teenagers, mainly between 8 and 15 years.

3.4.3. FEATURE PREVALENCE ANALYSIS

The code analyzes how frequently each clinical and histopathological feature appears in the dataset and visualizes this information using a horizontal bar chart. Initially, all input features are selected by excluding the age and target class columns. For each feature, the percentage of patients in whom the feature is present (i.e., values greater than zero) is calculated. This computation helps in understanding which symptoms or microscopic characteristics are common and which are relatively rare across the entire dataset.

The horizontal bar chart provides a clear comparative view of feature prevalence, where each bar represents the proportion of patients exhibiting a particular attribute. A vertical reference line at the 50% mark is included to distinguish between highly prevalent and less common features. The percentage values are also displayed at the end of each bar for precise interpretation. This analysis is important for identifying dominant characteristics, detecting low-variance features, and gaining insight into the overall structure of the dataset. Such understanding supports better feature selection, improves model learning, and enhances the interpretability of dermatological disease classification.

CODE:

```
feature_cols = [col for col in df_clean.columns if col not in ['age', 'class']]

# Calculate percentage of non-zero values for each feature
nonzero_pct = [(df_clean[col] > 0).sum() / len(df_clean) * 100 for col in feature_cols]

fig, ax = plt.subplots(figsize=(14, 8))
y_pos = np.arange(len(feature_cols))
bars = ax.barh(y_pos, nonzero_pct, color='steelblue', alpha=0.8)
ax.set_yticks(y_pos)
ax.set_yticklabels(feature_cols, fontsize=9)
ax.set_xlabel('Percentage of Patients with Feature Present (%)')
ax.set_title('Feature Prevalence Across All Patients')
ax.axvline(50, color='red', linestyle='--', alpha=0.5, label='50% threshold')

for bar, pct in zip(bars, nonzero_pct):
    ax.text(bar.get_width() + 1, bar.get_y() + bar.get_height()/2, f'{pct:.0f}%',
            va='center', fontsize=8)

plt.tight_layout()
plt.show()
```

OUTPUT:

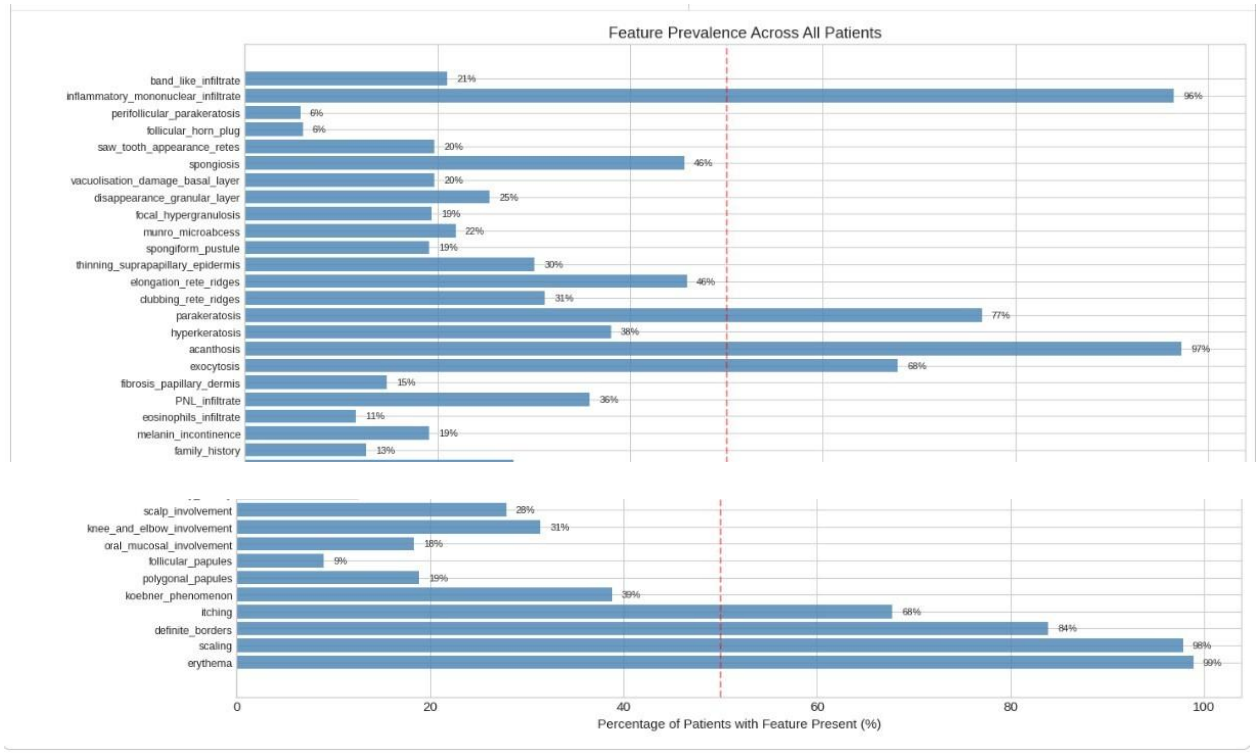


Fig 3.4.3.Feature Prevalence

INSIGHTS:

- The most dominant features are erythema, scaling, acanthosis, and inflammatory mononuclear infiltrate, all appearing in a very high percentage of patients (around 80–99%), indicating they are key characteristics across most cases.
- Features like parakeratosis, exocytosis, itching, and definite borders are also quite common, generally present in more than half of the patients, making them important supporting indicators.
- Moderately occurring features include hyperkeratosis, koebner phenomenon, PNL infiltrate, elongation of rete ridges, and knee/elbow involvement, which appear in roughly 30–50% of cases.
- Several features such as follicular papules, eosinophils infiltrate, family history, and oral mucosal involvement show low prevalence, typically below 20%, suggesting they are less common and may be condition-specific.

3.4.4. DISEASE-WISE FEATURE INTENSITIVE ANALYSIS USING HEATMAP

The below code computes the average value of each clinical and histopathological feature for every dermatological disease class and represents the results using a heatmap for better visualization. The dataset is first grouped based on the target variable (class), and the mean of each feature is calculated within each disease category. This provides a disease-specific profile that indicates how strongly a particular feature is expressed in different skin conditions. The numeric class labels are then replaced with their corresponding disease names to improve readability and interpretability of the visualization.

The heatmap displays features along the vertical axis and disease types along the horizontal axis, while the color intensity represents the average magnitude of each feature for a given disease. Warmer colors indicate higher average values, meaning the feature is more prominent for that disease, whereas lighter shades indicate lower presence. This visual analysis helps in identifying distinctive patterns and characteristic attributes associated with each dermatological condition. It also enables easy comparison between diseases and supports medical understanding by highlighting which clinical or histopathological findings are most influential for specific disease classes. Such insights are valuable for feature importance analysis, improving model performance, and enhancing the interpretability of the classification system.

CODE:

```
feature_means = df_clean.groupby('class')[feature_cols].mean()
feature_means.index = [disease_names[i] for i in feature_means.index]

fig, ax = plt.subplots(figsize=(16, 8))
sns.heatmap(feature_means.T, annot=False, cmap='YlOrRd', linewidths=0.5, ax=ax)
ax.set_title('Average Feature Intensity by Disease Type', fontsize=14)
ax.set_xlabel('Disease Type')
ax.set_ylabel('Features')
plt.tight_layout()
plt.show()
```

OUTPUT:

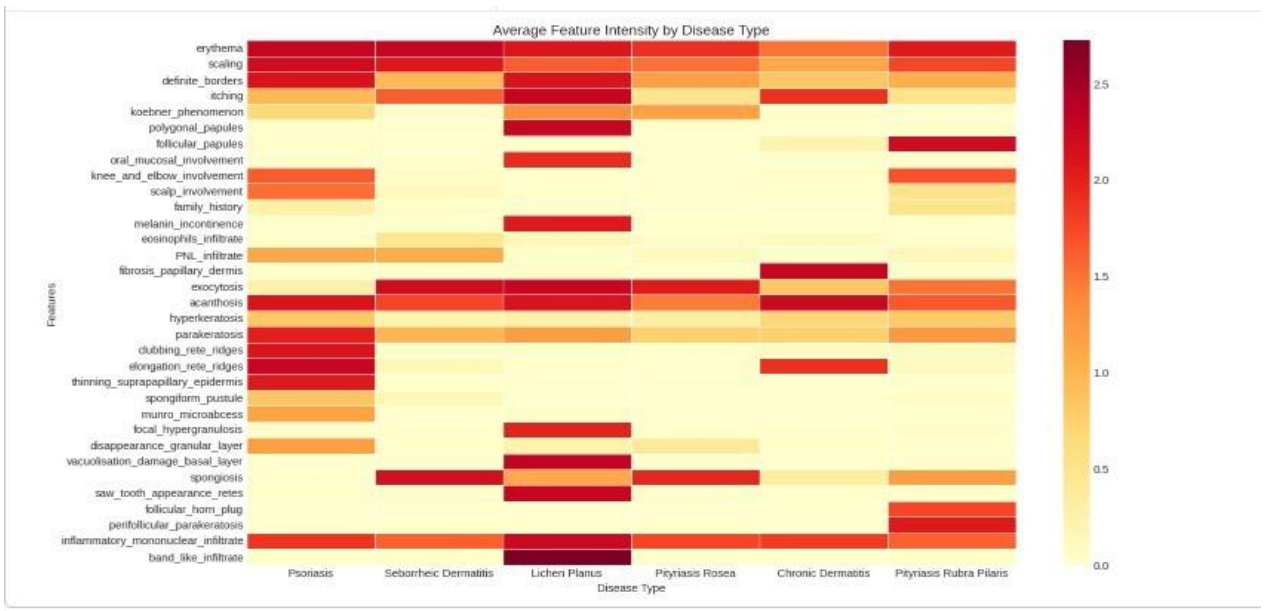


Fig 3.4.4.Feature Intensity

INSIGHTS:

- Erythema and scaling are highly intense across all diseases, making them common features.
- Psoriasis and Lichen Planus show strong intensity in acanthosis and hyperkeratosis.
- Lichen Planus is distinct with features like polygonal papules and oral involvement.
- Chronic Dermatitis shows higher fibrosis and structural changes.

- Pityriasis Rubra Pilaris stands out with follicular papules and joint involvement, while Seborrheic Dermatitis shows mostly moderate intensity.

3.4.5. FEATURE CORRELATION ANALYSIS USING HEATMAP

This code snippet is used to analyse the relationships between different clinical, histopathological, and demographic features in the dermatology dataset through a correlation matrix. Initially, the correlation values are computed using the .corr() function for all input features along with the age attribute. The correlation matrix quantifies the strength and direction of the linear relationship between every pair of variables, with values ranging from –1 to +1. A value close to +1 indicates a strong positive correlation, a value close to –1 indicates a strong negative correlation, and values near 0 indicate little or no linear relationship.

To enhance the clarity of the visualization and avoid redundancy, a mask is applied to hide the upper triangular portion of the matrix because correlation is symmetrical. The heatmap is then plotted using the seaborn library with a diverging colour palette (RdBu_r), where contrasting colours represent positive and negative correlations. The square grid structure, controlled linewidths, and fixed scale from –1 to +1 make the plot more interpretable and visually balanced.

This analysis helps in identifying highly correlated features, which is important for understanding multicollinearity within the dataset. Detecting such relationships is useful during feature selection, as strongly correlated attributes may carry similar information and can influence model performance. Additionally, the correlation heatmap provides deeper insight into how different dermatological characteristics interact with each other and with age, thereby supporting better data understanding, improved model design, and more reliable predictive analysis.

3.4.6. IDENTIFICATION OF TOP CORRELATED FEATURE PAIRS

This code segment is used to extract and identify the most strongly correlated feature pairs from the previously generated correlation matrix. Instead of only visualising the correlations through a heatmap, this step performs a systematic numerical analysis to determine which specific attributes have the highest relationship with one another. The code iterates through the

lower triangular portion of the correlation matrix to avoid duplicate pairs and self-correlation. For each unique pair of features, the corresponding correlation value is stored in a list along with the feature names. After collecting all possible feature combinations, the list is sorted in descending order based on the absolute value of the correlation coefficient. Using the absolute value ensures that both strong positive and strong negative correlations are considered equally important. From this sorted list, the top ten most highly correlated feature pairs are selected and displayed in a formatted manner.

This analysis plays a significant role in understanding multicollinearity within the dataset. Highly correlated features often carry similar information, which can influence machine learning models by increasing redundancy and potentially leading to overfitting. By identifying these feature pairs, the study gains deeper insight into relationships among clinical and histopathological attributes, which supports better feature selection, improves model efficiency, and enhances interpretability of the dermatological disease classification process.

CODE:

```
corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_pairs.append((corr_matrix.columns[i], corr_matrix.columns[j],
                           corr_matrix.iloc[i, j]))

corr_pairs_sorted = sorted(corr_pairs, key=lambda x: abs(x[2]), reverse=True)

print("Top 10 Most Correlated Feature Pairs:")
print("-" * 70)
for f1, f2, corr in corr_pairs_sorted[:10]:
    print(f"{f1:35} <-> {f2:35}: {corr:.3f}")
```

OUTPUT:

```
Top 10 Most Correlated Feature Pairs:
----------------------------------------------------------------------
melanin_incontinence                <-> vacuolisation_damage_basal_layer   : 0.942
vacuolisation_damage_basal_layer    <-> saw_tooth_appearance_retes         : 0.938
vacuolisation_damage_basal_layer    <-> band_like_infiltrate               : 0.938
follicular_horn_plug                <-> perifollicular_parakeratosis       : 0.929
saw_tooth_appearance_retes          <-> band_like_infiltrate               : 0.929
melanin_incontinence                <-> band_like_infiltrate               : 0.917
polygonal_papules                   <-> vacuolisation_damage_basal_layer   : 0.912
focal_hypergranulosis               <-> vacuolisation_damage_basal_layer   : 0.910
polygonal_papules                   <-> melanin_incontinence               : 0.907
polygonal_papules                   <-> band_like_infiltrate               : 0.906
```

3.4.7. FEATURE IMPORTANCE ANALYSIS USING POINT-BISERIAL CORRELATION

This code segment determines the most influential features for each dermatological disease class using the point-biserial correlation method. In this process, the multiclass target variable is converted into a binary format for each disease, where the selected disease is marked as 1 and all other diseases are marked as 0. This transformation enables the measurement of how strongly each clinical and histopathological feature is associated with the presence or absence of a particular disease.

For every disease category, the point-biserial correlation coefficient is calculated between the binary target and each input feature, including age. The computed correlation values indicate both the strength and the direction of the relationship. A positive correlation signifies that the feature is more prominent in patients with that disease, while a negative correlation indicates that the feature is less associated with that disease.

All correlation values are then organized into a structured DataFrame, allowing easy comparison of feature importance across different diseases. From this table, the top five features with the highest absolute correlation values are extracted for each disease and displayed. This step helps in identifying the most predictive attributes that contribute to distinguishing one skin disease from another.

This analysis significantly enhances the interpretability of the machine learning process. It provides medical insight into which symptoms and histopathological characteristics are most relevant for diagnosing specific dermatological conditions. Furthermore, it supports feature selection, reduces dimensionality, and improves the overall performance and reliability of the classification models by focusing on the most meaningful variables.

```
from scipy.stats import pointbiserialr

feature_importance_by_class = {}
for class_id in range(1, 7):
    binary_target = (df_clean['class'] == class_id).astype(int)
    correlations = {}
    for col in feature_cols + ['age']:
        corr, _ = pointbiserialr(binary_target, df_clean[col])
        correlations[col] = corr
    feature_importance_by_class[disease_names[class_id]] = correlations

importance_df = pd.DataFrame(feature_importance_by_class)

# Show top features for each disease
print("Top 5 Most Predictive Features per Disease (by correlation):")
print("=" * 60)
for disease in importance_df.columns:
    top5 = importance_df[disease].abs().nlargest(5)
    print(f"\n{disease}:")
    for feat, val in top5.items():
        actual_val = importance_df.loc[feat, disease]
        direction = "+" if actual_val > 0 else "-"
        print(f"  {direction} {feat}: {actual_val:.3f}")
```

OUTPUT:

```
Top 5 Most Predictive Features per Disease (by correlation):
============================================================

Psoriasis:
  + clubbing_rete_ridges: 0.914
  + thinning_suprapapillary_epidermis: 0.912
  + scalp_involvement: 0.740
  + elongation_rete_ridges: 0.725
  + munro_microabcess: 0.690

Seborrheic Dermatitis:
  + spongiosis: 0.480
  + eosinophils_infiltrate: 0.348
  + exocytosis: 0.336
  - elongation_rete_ridges: -0.319
  - koebner_phenomenon: -0.296

Lichen Planus:
  + band_like_infiltrate: 0.971
  + vacuolisation_damage_basal_layer: 0.960
  + saw_tooth_appearance_retes: 0.954
  + polygonal_papules: 0.947
  + melanin_incontinence: 0.941

Pityriasis Rosea:
  + spongiosis: 0.350
  - elongation_rete_ridges: -0.336
  - itching: -0.310
  - acanthosis: -0.280
  - clubbing_rete_ridges: -0.247

Chronic Dermatitis:
  + fibrosis_papillary_dermis: 0.933
  - scaling: -0.384
  - erythema: -0.348
  - definite_borders: -0.316
  + elongation_rete_ridges: 0.313

Pityriasis Rubra Pilaris:
  + perifollicular_parakeratosis: 0.953
  + follicular_horn_plug: 0.880
  + follicular_papules: 0.858
  - age: -0.413
  + family_history: 0.271
```

3.5. MODEL DEVELOPMENT

3.5.1Train–Test Split

This code segment is used to divide the dataset into two separate subsets: the training set and the testing set. The train_test_split() function from the Scikit-learn library is applied to the feature matrix **X** and the target variable **y**. In this project, 80% of the data is allocated for training the machine learning models, while the remaining 20% is reserved for testing their performance on unseen data. This separation is essential to evaluate how well the trained model generalizes to new samples and to prevent overfitting.

The parameter random_state=42 ensures reproducibility, meaning that the same split will be generated every time the code is executed. The stratify=y argument plays a crucial role in maintaining the original class distribution of the six dermatological diseases in both the training and testing datasets. Since medical datasets often require balanced representation of all disease categories, stratification helps in producing reliable and unbiased model evaluation.

After splitting the data, the code prints the number of samples in the training and testing sets along with the class distribution in the training data. This verification step confirms that the dataset has been properly divided and that all disease classes are proportionally represented, which is important for building a robust and accurate dermatological disease classification model.

CODE:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")
print(f"\nClass distribution in training set:")
print(y_train.value_counts().sort_index())
```

```
X = df_clean.drop('class', axis=1)
y = df_clean['class']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"\nTarget classes: {sorted(y.unique())}")
```

OUTPUT:

```
Training set: 292 samples
Test set: 74 samples

Class distribution in training set:
class
1    89
2    49
3    57
4    39
5    42
6    16
Name: count, dtype: int64
```

3.5.2.FEATURE SCALING USING STANDARDSCALER

This code segment performs feature scaling using the **StandardScaler** technique to standardize the numerical feature values before training the machine learning models. Feature scaling is an important preprocessing step because many machine learning algorithms, such as Support Vector Machine, K-Nearest Neighbors, and Logistic Regression, are sensitive to the magnitude of input values. If the features are on different scales, variables with larger ranges can dominate the learning process and negatively affect model performance.

In this implementation, the scaler is first fitted on the training data using fit_transform(). This step calculates the mean and standard deviation of each feature from the training set and transforms the data so that it has a mean of 0 and a standard deviation of 1. The same transformation is then applied to the test data using transform() without refitting the scaler. This is a crucial practice because it prevents information from the test set from leaking into the training process, thereby ensuring a fair evaluation of the model.

To demonstrate the effect of scaling, the code prints the mean and standard deviation of the **age** feature before and after scaling. Before scaling, the feature has its original statistical distribution, whereas after scaling it is standardized to a mean close to 0 and a standard deviation close to 1. This transformation ensures that all features contribute equally to the model training process, improves convergence speed, and enhances the overall performance and stability of the classification algorithms used in the dermatological disease prediction system.

CODE:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Features scaled using StandardScaler (mean=0, std=1)")
print(f"\nExample - Age before scaling: mean={X_train['age'].mean():.2f}, std={X_train['age'].std():.2f}")
print(f"Example - Age after scaling: mean={X_train_scaled[:, -1].mean():.2f}, std={X_train_scaled[:, -1].std():.2f}")
```

OUTPUT:

```
Features scaled using StandardScaler (mean=0, std=1)

Example - Age before scaling: mean=36.67, std=15.56
Example - Age after scaling: mean=-0.00, std=1.00
```

3.5.3.MODEL DECLARATION

This code segment defines the set of supervised machine learning algorithms that are used for dermatological disease classification in the proposed system. A dictionary named models is created in which the model names act as keys and the corresponding initialized machine learning algorithms act as values. The models included in this study are Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Gradient Boosting, and Support Vector Machine (SVM with RBF kernel). These algorithms are selected because they represent different learning approaches such as linear models, distance-based learning, tree-based methods, ensemble techniques, and kernel-based classification, enabling a comprehensive comparative analysis.

Each model is initialized with appropriate parameters. For example, random_state=42 is used in several algorithms to ensure reproducibility of results, max_iter=1000 is specified for Logistic Regression to allow sufficient iterations for convergence, and n_estimators=100 is used in ensemble methods to build multiple decision trees for improved predictive performance. After defining the models, a loop is used to print the names of all algorithms that will be evaluated.

This step forms the foundation for the model training and comparison phase. By organizing multiple algorithms in a structured format, the system enables systematic training, performance evaluation, and identification of the best-performing model for accurate and reliable dermatological disease classification.

CODE:

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state=42),
    'SVM (RBF Kernel)': SVC(kernel='rbf', random_state=42)
}

print("Models to evaluate:")
for name in models:
    print(f"  - {name}")
```

OUTPUT:

```
Models to evaluate:
  - Logistic Regression
  - K-Nearest Neighbors
  - Decision Tree
  - Random Forest
  - Gradient Boosting
  - SVM (RBF Kernel)
```

3.6. MODEL EVALUATION

3.6.1. Stratified K-Fold Cross-Validation for Model Evaluation

This section describes the implementation of Stratified K-Fold Cross-Validation to evaluate the performance and reliability of multiple machine learning models for dermatological disease classification. A StratifiedKFold object is created with five folds, ensuring that the dataset is divided into equal subsets while maintaining the original class distribution in each fold. The shuffle parameter is enabled to randomly mix the data before splitting, and a fixed random state is set to guarantee reproducibility of results. Stratification is particularly important in medical datasets, as it preserves class balance and prevents biased evaluation caused by uneven class representation.

To systematically compare different algorithms, a results dictionary is used to store cross-validation metrics for each model. Before training, the code checks whether feature scaling is required. Algorithms such as Logistic Regression, K-Nearest Neighbors, and Support Vector Machine are sensitive to feature magnitudes and are therefore trained on standardized data. In contrast, tree-based models such as Decision Tree, Random Forest, and Gradient Boosting are scale-invariant and are trained on the original dataset. The cross_val_score() function is applied using accuracy as the evaluation metric, and the accuracy for each fold is calculated.

Finally, the mean accuracy and standard deviation are computed from the fold-wise results to measure overall predictive performance and model stability. The mean accuracy represents how well the model performs on average, while the standard deviation indicates consistency across different subsets of data. Individual fold accuracies are also printed to provide a detailed view of performance variation. This validation approach eliminates reliance on a single train–test split, enables fair comparison among models, and ensures the selection of the most accurate and stable classifier for dermatological disease prediction.

CODE:

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = {}
print("\n" + "=" * 60)
print("CROSS-VALIDATION RESULTS (5-Fold Stratified)")
print("=" * 60)

for name, model in models.items():
    # Use scaled data for models that need it
    if name in ['Logistic Regression', 'K-Nearest Neighbors', 'SVM (RBF Kernel)']:
        X_cv = X_train_scaled
    else:
        X_cv = X_train

    # Cross-validation
    cv_scores = cross_val_score(model, X_cv, y_train, cv=cv, scoring='accuracy')
    results[name] = {
        'cv_mean': cv_scores.mean(),
        'cv_std': cv_scores.std(),
        'cv_scores': cv_scores
    }
    print(f"\n{name}:")
    print(f"  CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std()*2:.4f})")
    print(f"  Per-fold: {[f'{s:.3f}' for s in cv_scores]}")
```

OUTPUT:

```
============================================================
CROSS-VALIDATION RESULTS (5-Fold Stratified)
============================================================

Logistic Regression:
  CV Accuracy: 0.9760 (+/- 0.0171)
  Per-fold: ['0.983', '0.983', '0.983', '0.966', '0.966']

K-Nearest Neighbors:
  CV Accuracy: 0.9691 (+/- 0.0334)
  Per-fold: ['1.000', '0.949', '0.966', '0.966', '0.966']

Decision Tree:
  CV Accuracy: 0.9555 (+/- 0.0276)
  Per-fold: ['0.949', '0.966', '0.966', '0.931', '0.966']

Random Forest:
  CV Accuracy: 0.9725 (+/- 0.0278)
  Per-fold: ['0.983', '0.983', '0.948', '0.983', '0.966']

Gradient Boosting:
  CV Accuracy: 0.9657 (+/- 0.0379)
  Per-fold: ['0.966', '0.983', '0.931', '0.983', '0.966']

SVM (RBF Kernel):
  CV Accuracy: 0.9760 (+/- 0.0171)
  Per-fold: ['0.983', '0.983', '0.983', '0.966', '0.966']
```

3.6.2. Cross-Validation Accuracy Comparison

This section presents a comparative visualization of machine learning models using their mean cross-validation accuracy obtained from Stratified K-Fold evaluation. The primary objective is to display the performance of all trained algorithms in a clear and interpretable graphical format, enabling easier identification of the most effective model for dermatological disease classification. The model names, their corresponding mean accuracy scores, and standard deviations are extracted from the results dictionary. A bar chart is generated in which each bar represents a different algorithm, the height indicates the mean cross-validation accuracy, and error bars represent the standard deviation, reflecting the stability and consistency of each model across the folds.

The visualization is carefully designed to enhance readability and analytical clarity. The x-axis displays the model names, often rotated for better presentation, while the y-axis represents the cross-validation accuracy. The accuracy range is limited to focus on high-performing models, and a horizontal dashed line is drawn at the level of the highest mean accuracy to highlight the best-performing algorithm. Additionally, exact accuracy values are displayed above each bar to allow precise numerical comparison. This graphical representation transforms statistical results into an intuitive visual format, supporting transparent and evidence-based decision-making in healthcare-related machine learning projects.

Following the visualization, the best-performing model is identified programmatically using the mean cross-validation accuracy values. The max() function is applied to the results dictionary to determine which algorithm achieved the highest average accuracy across the stratified folds. The name of the top model and its corresponding accuracy are printed to ensure objective selection. After identifying the leading algorithm, the system proceeds with hyperparameter tuning—particularly for the Random Forest classifier due to its robustness, ability to handle high-dimensional medical data, resistance to noise, and built-in feature importance capabilities. This step ensures that the final model is not only statistically validated but also optimized for improved predictive accuracy and generalization on unseen dermatological data.

CODE:

```
fig, ax = plt.subplots(figsize=(12, 6))

model_names = list(results.keys())
means = [results[m]['cv_mean'] for m in model_names]
stds = [results[m]['cv_std'] for m in model_names]

x_pos = np.arange(len(model_names))
bars = ax.bar(x_pos, means, yerr=stds, capsize=5, color=plt.cm.Set2(np.linspace(0, 1, len(model_names))),
              edgecolor='black', alpha=0.8)

ax.set_xticks(x_pos)
ax.set_xticklabels(model_names, rotation=45, ha='right')
ax.set_ylabel('Cross-Validation Accuracy')
ax.set_title('Model Comparison: 5-Fold Cross-Validation Accuracy')
ax.set_ylim(0.8, 1.0)
ax.axhline(y=max(means), color='red', linestyle='--', alpha=0.5, label=f'Best: {max(means):.3f}')
ax.legend()

for bar, mean in zip(bars, means):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01, f'{mean:.3f}',
            ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```

OUTPUT:

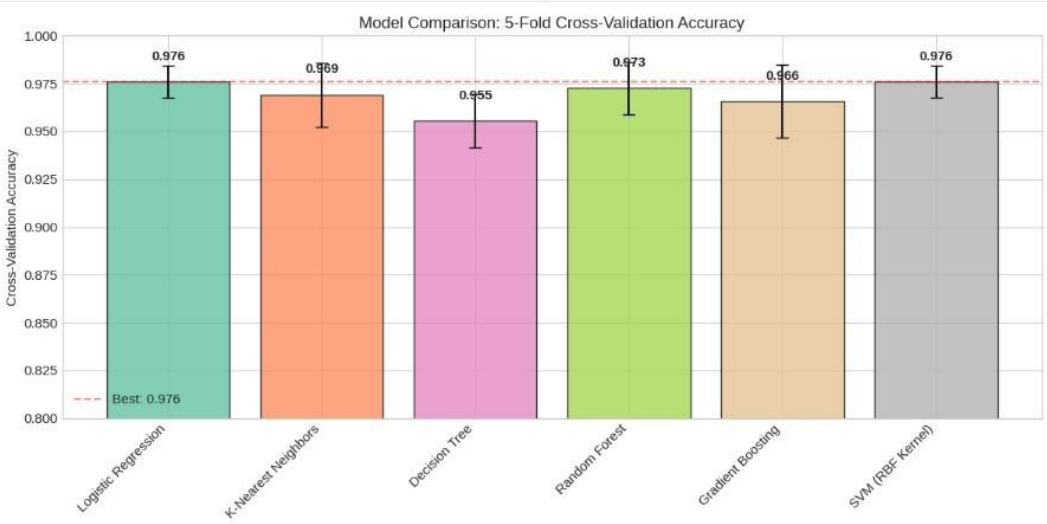


Fig 3.6.1.Model Comparison

3.7 MODEL OPTIMISATION

3.7.1. Hyperparameter Tuning

This section focuses on hyperparameter optimization for the Random Forest classifier using the Grid Search technique to achieve the best possible model performance. Hyperparameters are configuration settings that control how a machine learning algorithm learns from data, and tuning them is essential for improving accuracy, reducing overfitting, and enhancing generalization. A parameter grid is defined with multiple values for key Random Forest parameters, including the number of trees (n_estimators), maximum tree depth (max_depth), minimum samples required to split a node (min_samples_split), and minimum samples required at a leaf node (min_samples_leaf). These parameters directly influence the model's complexity, learning capacity, and robustness.

The GridSearchCV method is then applied to systematically evaluate all possible combinations of the specified hyperparameters. Each combination is assessed using 5-fold cross-validation with accuracy as the scoring metric, ensuring reliable and statistically validated performance measurement. The n_jobs = -1 setting enables parallel processing across all CPU cores, significantly reducing computation time, while the verbose option provides progress updates during the tuning process. This structured and automated search ensures that the model is optimized based on data-driven evaluation rather than default parameter settings.

After completion of the tuning process, the system identifies and outputs the best set of hyperparameters that achieved the highest cross-validation accuracy. The best_estimator_ attribute of the GridSearchCV object retrieves the optimized Random Forest model, which is then assigned to a variable for further prediction and evaluation. Printing the selected configuration provides transparency and reproducibility by clearly showing how the final model differs from the default setup. This step marks the transition from experimentation to deployment, ensuring that the final classifier is scientifically optimized, highly reliable, and well-suited for accurate dermatological disease classification on unseen data.

CODE:

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)

print(f"\nBest parameters: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.4f}")
```

```
best_rf = grid_search.best_estimator_
print("Best Random Forest Model:")
print(best_rf)
```

OUTPUT:

```
Fitting 5 folds for each of 108 candidates, totalling 540 fits

Best parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Best CV accuracy: 0.9794
```

```
Best Random Forest Model:
RandomForestClassifier(random_state=42)
```

3.8 MODEL TESTING

This module evaluates the performance of the **optimized Random Forest classifier** on the unseen test dataset to measure its real-world predictive capability. After completing hyperparameter tuning and selecting the best model, the predict() function is used to generate the predicted disease classes (y_pred) for the test data. Since the test dataset was not used during training or cross-validation, it provides an unbiased assessment of how well the model generalizes to new patient records.

To obtain a comprehensive performance analysis, multiple evaluation metrics are calculated. The **accuracy score** measures the overall proportion of correctly classified instances. However, accuracy alone may not fully reflect model performance in multiclass medical datasets; therefore, the **F1-score (weighted)** and **F1-score (macro)** are also computed. The weighted F1-score takes class imbalance into account by assigning importance based on the number of samples in each class, while the macro F1-score treats all classes equally and evaluates how well the model

performs across all dermatological disease categories. This dual evaluation ensures that the classifier is not biased toward any particular disease class.

The results are then displayed in a structured format, clearly presenting the final test accuracy and F1-scores. This step is crucial because it validates the effectiveness of the proposed system on unseen data, confirms the reliability of the optimized model, and demonstrates its suitability for accurate dermatological disease classification in practical healthcare scenarios.

CODE:

```
y_pred = best_rf.predict(X_test)

# Calculate metrics
test_accuracy = accuracy_score(y_test, y_pred)
test_f1_weighted = f1_score(y_test, y_pred, average='weighted')
test_f1_macro = f1_score(y_test, y_pred, average='macro')

print("=" * 60)
print("TEST SET PERFORMANCE")
print("=" * 60)
print(f"\nAccuracy: {test_accuracy:.4f}")
print(f"F1 Score (Weighted): {test_f1_weighted:.4f}")
print(f"F1 Score (Macro): {test_f1_macro:.4f}")
```

OUTPUT:

```
============================================================
TEST SET PERFORMANCE
============================================================

Accuracy: 0.9595
F1 Score (Weighted): 0.9595
F1 Score (Macro): 0.9545
```

3.8.2. CLASSIFICATION REPORT

The code generates a detailed performance evaluation of the trained classification model on the test dataset using the classification_report function from scikit-learn. It first prints a heading and a separator to clearly structure the output. Then, a list of target_names is created by mapping the numerical class labels to their corresponding disease names using a list comprehension. This ensures that the evaluation results are displayed with meaningful class labels instead of numeric values. Finally, the classification_report(y_test, y_pred, target_names=target_names) function

compares the true labels (y_test) with the predicted labels (y_pred) and computes key metrics such as precision, recall, F1-score, and support for each disease class, providing a comprehensive view of the model's classification performance.

The output shows that the model achieved an overall accuracy of 96% on the test data, indicating very strong predictive performance. Several classes, including Psoriasis, Lichen Planus, Chronic Dermatitis, and Pityriasis Rubra Pilaris, obtained perfect precision, recall, and F1-scores of 1.00, meaning all their instances were correctly classified without any errors. Seborrheic Dermatitis and Pityriasis Rosea show slightly lower recall and F1-scores, which suggests a small number of misclassifications for these categories. The macro average F1-score of 0.95 indicates that the model performs consistently across all classes, while the weighted average F1-score of 0.96 confirms that the model maintains high performance even when class distribution is considered. Overall, the results demonstrate that the model is accurate, balanced, and reliable for multi-class skin disease classification.

CODE:

```
print("\nDetailed Classification Report:")
print("-" * 60)
target_names = [disease_names[i] for i in sorted(y.unique())]
print(classification_report(y_test, y_pred, target_names=target_names))
```

OUTPUT :

```
Detailed Classification Report:
------------------------------------------------------------
                         precision    recall  f1-score   support

               Psoriasis       1.00      1.00      1.00        23
   Seborrheic Dermatitis       0.91      0.83      0.87        12
           Lichen Planus       1.00      1.00      1.00        15
         Pityriasis Rosea       0.82      0.90      0.86        10
      Chronic Dermatitis       1.00      1.00      1.00        10
Pityriasis Rubra Pilaris       1.00      1.00      1.00         4

                 accuracy                           0.96        74
                macro avg       0.95      0.96      0.95        74
             weighted avg       0.96      0.96      0.96        74
```

3.8.3. CONFUSION MATRIX

The code computes and visualizes the confusion matrix to evaluate how well the trained model classifies each disease category. First, confusion_matrix(y_test, y_pred) generates a matrix that compares the actual labels with the predicted labels. A figure with two subplots is then created to display both the raw and normalized versions of the confusion matrix. The first heatmap shows the absolute number of predictions for each class using integer formatting (fmt='d'), which helps identify how many samples were correctly and incorrectly classified. The second heatmap normalizes the confusion matrix by dividing each row by the total number of actual instances in that class, providing a proportion-based view of performance. The axis labels are shortened for better readability, and appropriate titles, axis labels, and color mapping are applied to make the visualization clear and informative.

The confusion matrix (counts) visually represents the number of correct and incorrect predictions for each disease class. The diagonal elements indicate correctly classified instances, and the high values along the diagonal confirm that the model performs very well across most classes. Any off-diagonal values represent misclassifications and help identify which diseases are being confused with others. The normalized confusion matrix provides a class-wise accuracy perspective, showing the percentage of correctly classified samples for each actual class. Values close to 1.00 along the diagonal indicate excellent recall for those classes, while lower values highlight classes where the model makes relatively more errors. Together, these plots demonstrate that the model has strong classification capability with minimal confusion between disease 8 categories, while also helping to pinpoint specific classes that may require further improvement.

From the confusion matrix, it is evident that the model performs exceptionally well for most of the disease classes. Psoriasis, Lichen Planus, Chronic Dermatitis, and Pityriasis Rubra Pilaris are classified with 100% accuracy, as all their samples lie on the diagonal with no misclassifications. Seborrheic Dermatitis shows two misclassified instances that were predicted as Pityriasis Rosea, resulting in a normalized accuracy of 0.83 for that class. Similarly, Pityriasis Rosea has one sample misclassified as Seborrheic Dermatitis, giving it a recall of 0.90. The normalized confusion matrix clearly highlights these class-wise performance differences in percentage form. Overall, the strong diagonal dominance in both matrices indicates that the

model has learned highly discriminative patterns and achieves very high classification accuracy with minimal confusion between disease categories.

CODE:

```
cm = confusion_matrix(y_test, y_pred)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Raw counts
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=[d[:12] for d in target_names],
            yticklabels=[d[:12] for d in target_names])
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')
axes[0].set_title('Confusion Matrix (Counts)')

# Normalized
cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues', ax=axes[1],
            xticklabels=[d[:12] for d in target_names],
            yticklabels=[d[:12] for d in target_names])
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')
axes[1].set_title('Confusion Matrix (Normalized)')

plt.tight_layout()
plt.show()
```

OUTPUT :

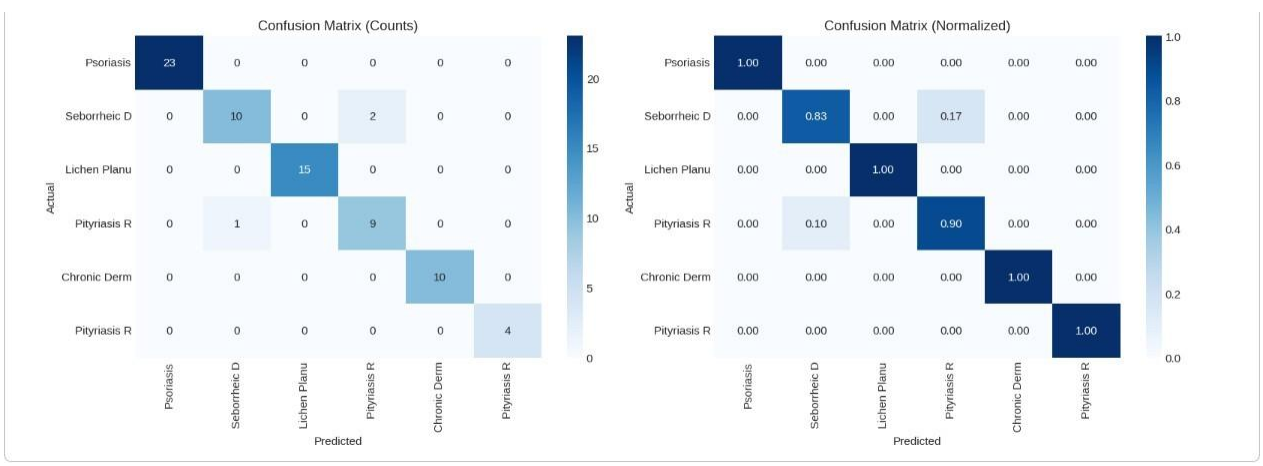


Fig 3.8.1.Correlation Map

3.8.4. FEATURE IMPORTANCE

This section focuses on computing and visualizing the feature importance of the optimized Random Forest model to understand which attributes contribute most to dermatological disease classification. A DataFrame is created by pairing each input feature with its corresponding importance score obtained from the model's feature_importances_ attribute, which is based on the Gini importance measure.

The features are sorted in ascending order so that the most influential variables appear at the top of the horizontal bar chart. For enhanced interpretability, a color gradient is applied, where darker shades represent higher importance values. A vertical dashed line is added to indicate the mean importance score, making it easier to distinguish features that contribute above average to the model's decision-making process.

The resulting visualization highlights the most significant clinical and histopathological attributes influencing predictions. Features with larger importance values play a stronger role in reducing impurity within decision trees and therefore have a greater impact on classification outcomes. Attributes positioned above the mean importance line are considered key predictors, while those with very low scores have minimal influence on performance. The plot improves model interpretability by clearly revealing which medical characteristics are most relevant for distinguishing between skin disease classes, and it also supports potential feature selection and dimensionality reduction in future model refinement.

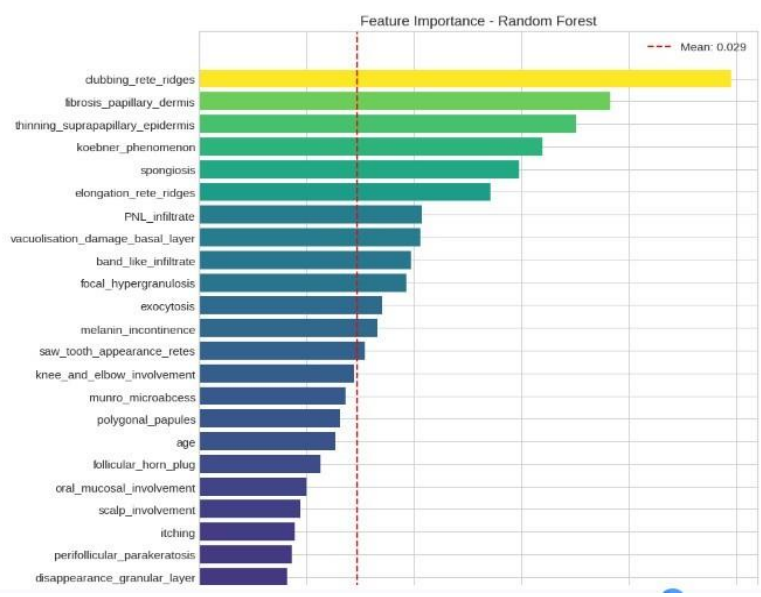
In addition, the top ten most important features are extracted and displayed in a structured format. Since the features were sorted in ascending order, the highest importance values are selected and reversed to present them in descending order. Each feature is also categorized as clinical, histopathological, or demographic (such as age), adding domain-level interpretability to the results. The output demonstrates that histopathological features—such as clubbing of rete ridges, fibrosis of the papillary dermis, thinning of the suprapapillary epidermis, Koebner phenomenon, and spongiosis—are among the strongest predictors in the model. This indicates that microscopic tissue characteristics contribute more significantly to accurate classification than visible clinical features, thereby enhancing transparency, supporting medical insight, and strengthening the reliability of the diagnostic framework.

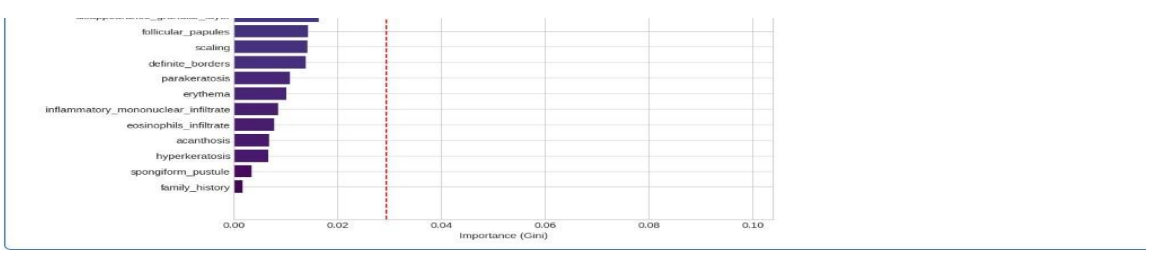
CODE:

```
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': best_rf.feature_importances_
}).sort_values('importance', ascending=True)

# Plot
fig, ax = plt.subplots(figsize=(10, 12))
colors = plt.cm.viridis(feature_importance['importance'] / feature_importance['importance'].max())
ax.barh(feature_importance['feature'], feature_importance['importance'], color=colors)
ax.set_xlabel('Importance (Gini)')
ax.set_title('Feature Importance - Random Forest')
ax.axvline(x=feature_importance['importance'].mean(), color='red', linestyle='--',
           label=f'Mean: {feature_importance["importance"].mean():.3f}')
ax.legend()
plt.tight_layout()
plt.show()
```

```
top_features = feature_importance.tail(10)
print("Top 10 Most Important Features:")
print("=" * 50)
for i, (_, row) in enumerate(top_features.iloc[::-1].iterrows(), 1):
    feat_type = "Clinical" if row['feature'] in clinical_features else "Histopath" if row['feature'] in histopathological_features else "Demo"
    print(f"{i:2}. {row['feature']:40} ({feat_type:10}): {row['importance']:.4f}")
```

OUTPUT:





```
Top 10 Most Important Features:
==================================================
 1. clubbing_rete_ridges                    (Histopath ): 0.0990
 2. fibrosis_papillary_dermis               (Histopath ): 0.0766
 3. thinning_suprapapillary_epidermis       (Histopath ): 0.0702
 4. koebner_phenomenon                      (Clinical  ): 0.0639
 5. spongiosis                              (Histopath ): 0.0596
 6. elongation_rete_ridges                  (Histopath ): 0.0541
 7. PNL_infiltrate                          (Histopath ): 0.0414
 8. vacuolisation_damage_basal_layer        (Histopath ): 0.0411
 9. band_like_infiltrate                    (Histopath ): 0.0394
10. focal_hypergranulosis                   (Histopath ): 0.0385
```

Fig 3.8.2.Feature Significance

3.9.MODEL COMPARISON

This code performs the final performance comparison of all trained machine learning models by training them on the training dataset and evaluating them on the test dataset. It first iterates through each model in the dictionary and applies the appropriate preprocessing: scaled features are used for models that are sensitive to feature magnitude (Logistic Regression, KNN, and SVM), while the remaining models are trained on the original data. For every model, key evaluation metrics such as cross-validation mean accuracy, cross-validation standard deviation, test accuracy, weighted F1-score, and macro F1-score are collected and stored in a list. In addition to the baseline models, the tuned Random Forest model obtained from GridSearchCV is also appended with its best cross-validation score and test performance. The results are then converted into a pandas DataFrame, sorted in descending order based on test accuracy, indexed properly, and displayed as a structured comparison table titled *Final Model Comparison*.

The output table provides a comprehensive comparison of all models based on their generalization performance. The SVM with RBF kernel achieved the highest test accuracy (≈97.3%) along with the best weighted and macro F1-scores, indicating superior classification performance across all disease classes. Logistic Regression and both versions of the Random Forest model also performed very well with test accuracy around 95.9%, showing stable and consistent results. Decision Tree and Gradient Boosting achieved moderate performance, while K-Nearest Neighbors recorded the lowest test accuracy (≈90.5%), suggesting it is less suitable for this dataset. The small difference between cross-validation accuracy and test accuracy for the

top models indicates good generalization and minimal overfitting. Overall, the comparison demonstrates that the SVM model is the most effective for this skin disease classification task, while the tuned Random Forest remains a strong and interpretable alternative.

CODE:

```
final_results = []

for name, model in models.items():
    if name in ['Logistic Regression', 'K-Nearest Neighbors', 'SVM (RBF Kernel)']:
        model.fit(X_train_scaled, y_train)
        pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        pred = model.predict(X_test)

    final_results.append({
        'Model': name,
        'CV Accuracy': results[name]['cv_mean'],
        'CV Std': results[name]['cv_std'],
        'Test Accuracy': accuracy_score(y_test, pred),
        'Test F1 (Weighted)': f1_score(y_test, pred, average='weighted'),
        'Test F1 (Macro)': f1_score(y_test, pred, average='macro')
    })

# Add tuned RF
final_results.append({
    'Model': 'Random Forest (Tuned)',
    'CV Accuracy': grid_search.best_score_,
    'CV Std': 0,
    'Test Accuracy': test_accuracy,
    'Test F1 (Weighted)': test_f1_weighted,
    'Test F1 (Macro)': test_f1_macro
})

results_df = pd.DataFrame(final_results).sort_values('Test Accuracy', ascending=False)
results_df.index = range(1, len(results_df) + 1)
print("\n" + "=" * 90)
```

```
print("FINAL MODEL COMPARISON")
print("=" * 90)
results_df
```

OUTPUT:

```
==========================================================================================
FINAL MODEL COMPARISON
==========================================================================================
```

	Model	CV Accuracy	CV Std	Test Accuracy	Test F1 (Weighted)	Test F1 (Macro)
1	SVM (RBF Kernel)	0.975979	0.008543	0.972973	0.972973	0.969697
2	Random Forest	0.972531	0.013881	0.959459	0.959543	0.954451
3	Logistic Regression	0.975979	0.008543	0.959459	0.960642	0.957362
4	Random Forest (Tuned)	0.979369	0.000000	0.959459	0.959543	0.954451
5	Decision Tree	0.955465	0.013798	0.932432	0.929245	0.887832
6	Gradient Boosting	0.965693	0.018942	0.932432	0.930944	0.912469
7	K-Nearest Neighbors	0.969141	0.016681	0.905405	0.908094	0.903428

3.10.RESULTS

This section of the code generates a summary of the overall project findings by extracting key results from the previously computed model comparison and feature importance outputs. It identifies the best-performing model based on the highest test accuracy and prints its corresponding performance metrics, including test accuracy and weighted F1-score. It then reports the main dataset characteristics such as the total number of samples, number of input features (clinical, histopathological, and demographic), number of target classes, and the presence of class imbalance. Next, the code retrieves and displays the top predictive features from the Random Forest feature importance analysis, highlighting the variables that contributed most to disease classification. Finally, it prints the clinical implications and limitations of the study in a structured format, making the output suitable for direct inclusion in a project report.

The output shows that the SVM with RBF kernel achieved the best performance, with a test accuracy of approximately 97.30% and a high weighted F1-score, indicating excellent and well-balanced classification across all six skin disease categories. The dataset consists of 366 samples and 34 input features, combining clinical observations, histopathological findings, and patient age, with only mild class imbalance, which supports reliable model training. The most influential predictive features—such as clubbing of rete ridges, fibrosis of the papillary dermis, thinning of the suprapapillary epidermis, Koebner phenomenon, and spongiosis—are medically relevant attributes, showing that the model's decisions align with real dermatological diagnostic patterns. Clinically, this demonstrates that machine learning can effectively assist in differentiating visually similar skin diseases and has strong potential for automated screening and decision support. However, the results are constrained by the relatively small dataset size, the need for imputation of missing values, and the reliance on biopsy-based histopathological features, meaning the system should be used as a support tool rather than a replacement for expert clinical judgment.

CODE:

```
print("=" * 70)
print("SUMMARY & KEY FINDINGS")
print("=" * 70)

best_test_model = results_df.iloc[0]['Model']
best_test_acc = results_df.iloc[0]['Test Accuracy']

print(f"""
1. BEST MODEL: {best_test_model}
   - Test Accuracy: {best_test_acc:.2%}
   - F1 Score (Weighted): {results_df.iloc[0]['Test F1 (Weighted)']:.4f}

2. DATASET CHARACTERISTICS:
   - Total samples: {len(df_clean)}
   - Features: {X.shape[1]} (11 clinical + 22 histopathological + 1 demographic)
   - Classes: 6 skin diseases
   - Mild class imbalance present

3. TOP PREDICTIVE FEATURES:""")

for i, row in feature_importance.tail(5).iloc[::-1].iterrows():
    print(f"   - {row['feature']}: {row['importance']:.4f}")

print(f"""
4. CLINICAL IMPLICATIONS:
   - Machine learning can effectively differentiate between similar-looking skin diseases
   - Both clinical and histopathological features contribute to diagnosis
   - Model could assist dermatologists in differential diagnosis
   - High accuracy ({best_test_acc:.0%}+) suggests reliable automated screening potential

5. LIMITATIONS:
   - Small dataset (366 samples) - may not generalize to all populations
   - Missing values required imputation
   - Requires histopathological data which needs biopsy
   - Model should support, not replace, clinical judgment
""")
```

OUTPUT:

```
======================================================================
SUMMARY & KEY FINDINGS
======================================================================

1. BEST MODEL: SVM (RBF Kernel)
   - Test Accuracy: 97.30%
   - F1 Score (Weighted): 0.9730

2. DATASET CHARACTERISTICS:
   - Total samples: 366
   - Features: 34 (11 clinical + 22 histopathological + 1 demographic)
   - Classes: 6 skin diseases
   - Mild class imbalance present

3. TOP PREDICTIVE FEATURES:
   - clubbing_rete_ridges: 0.0990
   - fibrosis_papillary_dermis: 0.0766
   - thinning_suprapapillary_epidermis: 0.0702
   - koebner_phenomenon: 0.0639
   - spongiosis: 0.0596

4. CLINICAL IMPLICATIONS:
   - Machine learning can effectively differentiate between similar-looking skin diseases
   - Both clinical and histopathological features contribute to diagnosis
   - Model could assist dermatologists in differential diagnosis
   - High accuracy (97%+) suggests reliable automated screening potential

5. LIMITATIONS:
   - Small dataset (366 samples) - may not generalize to all populations
   - Missing values required imputation
   - Requires histopathological data which needs biopsy
   - Model should support, not replace, clinical judgment
```

4. STREAMLIT WEB INTERFACE

The web application interface of the proposed dermatological disease classification system was developed using Streamlit, which enables the creation of interactive data science applications directly from Python scripts. The purpose of this interface is to transform the trained machine learning model into a user-friendly system that can be easily accessed and utilized by users without requiring programming knowledge. The application integrates data visualization, prediction functionality, and model interpretability into a single platform. It consists of three main components: a Dashboard, a Prediction Interface, and an Explainability Module. The dashboard provides an overview of the dataset through interactive charts and statistical summaries, allowing users to understand patterns such as disease distribution, feature importance, and demographic trends. The prediction page allows users to input relevant clinical and histopathological attributes of a patient, which are then processed by the trained classification model implemented using scikit-learn to generate a predicted dermatological disease class. Additionally, the application incorporates explainable artificial intelligence techniques using SHAP to interpret model predictions by displaying the contribution of each feature toward the final decision. This integration improves transparency and helps users understand the reasoning behind predictions rather than treating the model as a black-box system. Overall, the Streamlit web application acts as an interactive decision-support interface that combines data exploration, automated disease prediction, and model interpretability in a simple and efficient environment, making the system more accessible and practical for real-world use.

4.1. DASHBOARD MODULE

The Dashboard Module serves as the primary analytical interface of the web application, providing an interactive overview of the dermatology dataset and key insights generated from exploratory data analysis. Developed using Streamlit, the dashboard presents important summary statistics such as the total number of selected patients, average patient age, the most common disease category, and the overall diagnostic accuracy of the trained model. It also includes interactive visualizations that help users understand the distribution and relationships within the dataset. For instance, the disease breakdown chart visually represents the proportion of each dermatological condition, enabling quick identification of the most prevalent diseases. The dimensionality overlap visualization based on Principal Component Analysis (PCA) helps illustrate how different disease classes are distributed in a

reduced feature space, making it easier to observe patterns and separability among classes. Additionally, the dashboard includes filtering options that allow users to dynamically select or remove specific disease categories, instantly updating the visualizations to reflect the filtered data. These visualizations are generated using Python data analysis libraries such as pandas and Plotly, ensuring interactive and responsive charts. Overall, the dashboard module functions as an exploratory and monitoring tool that allows users to gain meaningful insights into the dataset, understand disease distribution patterns, and observe the performance characteristics of the machine learning model before proceeding to prediction or model explanation tasks.

4.2. PREDICTION MODULE

The Prediction Module functions as the core component of the application where real-time disease classification is performed based on user-provided inputs. This interface allows users to enter key clinical attributes such as patient age, itching severity, erythema, polygonal papules, scaling, and family history through interactive sliders and input fields. These inputs represent the same clinical and histopathological features that were used to train the machine learning model. Once the user provides the required parameters, the system processes the inputs and sends them to the trained classification model developed using scikit-learn. The model then predicts the most probable dermatological condition and displays the result as the predicted diagnosis along with a confidence score, indicating the probability of the prediction. In addition to the predicted class, the module also presents a visual representation of the output confidence distribution, which shows the probability scores for all possible disease classes. This graphical representation helps users understand how strongly the model supports one diagnosis compared to others. The prediction interface is implemented within an interactive environment using Streamlit, enabling real-time inference without requiring complex technical operations from the user. Overall, the prediction module transforms the trained machine learning system into a practical decision-support tool that allows users to simulate patient cases and obtain automated dermatological disease predictions quickly and efficiently

4.3. EXPLAINABILITY AI MODULE

The Model Explainability (SHAP) Module is designed to provide transparency and interpretability for the machine learning predictions generated by the system. In many machine learning applications, models often function as “black boxes,” making it difficult to understand how specific inputs

influence the final prediction. To address this issue, the application integrates explainable artificial intelligence techniques using SHAP (SHapley Additive exPlanations). This module provides both global and local explanations of the trained model's behavior. The global feature importance visualization shows the overall contribution of different clinical and histopathological features to the prediction process, highlighting the most influential attributes used by the model. These importance values are derived from the trained scikit-learn Random Forest model, which evaluates how each feature contributes to decision-making across the entire dataset. In addition, the module includes a local interpretation panel, where the prediction for an individual patient case is analyzed in detail. The SHAP visualization illustrates how each feature either increases or decreases the probability of a specific disease prediction, thereby explaining the reasoning behind the model's decision. This interpretability layer helps users understand the influence of key dermatological indicators such as thinning of the suprapapillary epidermis, clubbing of rete ridges, and Koebner phenomenon on the final prediction. The module is implemented within an interactive environment using Streamlit, allowing users to explore model explanations dynamically. Overall, the explainability module enhances the reliability and trustworthiness of the system by making the model's decision-making process transparent and understandable.

Fig.4.1.Streamlit Interface

5. CONCLUSION

5.1. CONCLUSION

This project presents a comprehensive and analytically strong machine learning framework for dermatological disease classification, successfully addressing the limitations of traditional diagnostic approaches and previously implemented machine learning systems. Dermatological diseases often exhibit overlapping clinical and histopathological features, making manual diagnosis complex and highly dependent on expert interpretation. By integrating data-driven techniques into the diagnostic process, this project demonstrates how machine learning can enhance accuracy, consistency, and efficiency in disease classification while reducing reliance on subjective judgment.

The system was developed using a structured and systematic workflow. It began with thorough data preprocessing, including handling missing values, correcting data types, and ensuring data quality. Exploratory Data Analysis (EDA) was conducted to understand feature distributions, class balance, and correlations among attributes, providing meaningful insights into disease patterns. A comparative analysis of multiple supervised machine learning algorithms—such as Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, Gradient Boosting, and Support Vector Machine—was performed to identify the most suitable model for classification. Unlike many existing approaches that rely on a single algorithm, this multi-model comparison ensured fairness, transparency, and robustness in model selection.

To enhance reliability and minimize bias, Stratified K-Fold Cross-Validation was implemented, preserving class distribution across folds and providing statistically stable performance estimates. Hyperparameter optimization using Grid Search further improved the predictive capability of the selected model by systematically identifying the best parameter combinations rather than relying on default settings. Model evaluation was conducted using multiple performance metrics, including Accuracy, Precision, Recall, F1-Score, and Confusion Matrix, ensuring a comprehensive and meaningful assessment beyond simple accuracy measurement.

An important contribution of this project is the integration of feature importance analysis, which improved interpretability by identifying the most influential clinical and histopathological

attributes affecting disease prediction. This transparency is particularly significant in healthcare applications, where understanding the reasoning behind predictions is essential for gaining clinical trust. The interpretability component not only strengthens confidence in the model's outputs but also provides valuable medical insights into the factors contributing most to dermatological disease classification.

In conclusion, the project successfully demonstrates that a structured, validated, and optimized machine learning framework can significantly improve dermatological disease classification. By combining preprocessing, comparative model evaluation, robust validation, hyperparameter tuning, and interpretability analysis, the proposed system provides a reliable and analytically rich diagnostic support tool. This work lays a strong foundation for future enhancements, including real-time clinical decision support systems, deployment as a web-based healthcare application, and integration with hospital databases. Ultimately, the project highlights the transformative potential of machine learning in improving healthcare decision-making and supporting accurate, efficient, and explainable medical diagnosis.

5.2. FUTURE ENHANCEMENT

1. Model Optimization and Ensemble Techniques:

- Experiment with advanced ensemble methods by combining algorithms such as Random Forest, Gradient Boosting, and Support Vector Machine to further improve classification accuracy.
- Apply advanced hyperparameter tuning techniques such as Bayesian Optimization or Random Search in addition to Grid Search to enhance model performance and efficiency.

2. Real-Time Clinical Decision Support System:

- Develop a real-time dermatological diagnosis system that allows clinicians to input patient data and receive instant predictions.
- Deploy the model as a web or mobile application and integrate it with hospital management systems for seamless clinical usage.

3. Adaptive Learning Mechanisms:

- Implement continuous learning models that can update automatically when new patient data becomes available, improving adaptability over time.
- Explore incremental learning techniques to ensure the model remains effective with evolving disease patterns and new clinical evidence.

4. Advanced Feature Engineering:

- Incorporate additional patient-related attributes such as genetic factors, lifestyle habits, environmental exposure, and medical history to enhance predictive capability.
- Apply dimensionality reduction techniques such as Principal Component Analysis (PCA) to optimize feature representation and reduce redundancy.

5. Handling Class Imbalance:

- Implement advanced resampling techniques such as SMOTE (Synthetic Minority Over-sampling Technique) or ADASYN to improve classification of minority disease classes.
- Explore cost-sensitive learning approaches to reduce misclassification of critical disease categories.

6. Explainable AI (XAI) Integration:

- Integrate explainability tools such as SHAP and LIME to provide instance-level explanations for individual predictions.
- Develop interactive dashboards to visualize feature contributions and improve transparency for medical professionals.

7. Image-Based Disease Classification:

- Extend the system by incorporating dermoscopic or skin lesion images using deep learning models such as Convolutional Neural Networks (CNNs).
- Develop a hybrid multimodal system that combines image data with structured clinical attributes for enhanced diagnostic accuracy.

8. Large-Scale and Multi-Center Validation:

- Train and validate the model using larger, multi-center datasets to improve generalization across diverse populations.
- Conduct real-world clinical validation studies to assess practical effectiveness and reliability.

9. Personalized Treatment Recommendation System:

- Expand the framework to include treatment suggestions based on predicted disease class and patient-specific characteristics.
- Develop risk prediction modules to support early detection and preventive healthcare strategies.

5.3. REFERENCES

1. Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science. Available at: https://archive.ics.uci.edu ,(Dermatology dataset source) .
2. Güvenir, H. A., Demiröz, G., & İlter, N. (1998). Learning differential diagnosis of erythemato-squamous diseases using voting feature intervals. *Artificial Intelligence in Medicine*, 13(3), 147–165.
3. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
4. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
5. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
6. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
7. Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 1137–1143.
8. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
9. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. (SHAP – Explainable AI)

5.4. BIBLIOGRAPHY

1. Dua, Dheeru and Graff, Casey. *UCI Machine Learning Repository*. Irvine, CA: University of California Irvine Machine Learning Repository, 2019.https://archive.ics.uci.edu
2. Ron Kohavi. "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection." In *International Joint Conference on Artificial Intelligence (IJCAI)*, 1995.
3. Leo Breiman. "Random Forests." *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
4. Corinna Cortes and Vladimir Vapnik. "Support-Vector Networks." *Machine Learning*, vol. 20, pp. 273–297, 1995.
5. Trevor Hastie, Robert Tibshirani, and Jerome H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.
6. Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
7. Python Software Foundation. *Python Programming Language*. https://www.python.org